Agentic Engineering

Realita, Strategi, dan Praktik
untuk Tim Produk & Bisnis Digital

👋 Perkenalan

https://www.zainfathoni.com/about

- 📍 Jember ➡️ Bandung ➡️ 🇸🇬 SG ➡️ Jogja
- 🛠️ Backend ➡️ Manager ➡️ Frontend ➡️ Fullstack
- 🤖 Agentic Engineering Practitioner
- 🤝 Talk ini untuk semua: dev, founder, marketer, dan produk

Agenda

1. **The Reality of AI-Assisted Development**
2. **Coverage & Control**
3. **Prompting & Interaction Strategy**
4. **Structured AI Coding**
5. **Maintaining Craftsmanship**
6. **Live Demo Results** 👀

🎬 Live Demo Setup

Jalankan Claude Code di terminal, lalu lanjut presentasi

```
claude "Tambahkan filter by category (Conferences /
 Podcasts / Meetups) ke halaman /talks di zainf.dev.
 Lihat pola filter di halaman /blog sebagai referensi."
```

⏳ Claude Code bekerja di background...

1) The Reality of AI-Assisted Development

🤖 Apa itu Agentic Engineering?

> "Agentic engineering: developing software with the assistance of coding agents that can write **and execute** code."
> — Simon Willison

- 👓 Agent bekerja dalam loop: tulis kode → jalankan → iterasi
- 🧠 Tugas kita: **specify** masalah, **verify** hasil, **iterate**
- ❌ Bukan "vibe coding" — bukan asal generate dan forget
- ✅ Ini yang kita lakukan hari ini dengan Claude Code

🤖 Peran AI di Setiap Fase Kerja Digital

Fase	Tools AI	Peran
🎨 **Design**	v0, Figma AI	Prototyping & landing page
📝 **Content**	ChatGPT, Claude	Copy, outline, dokumentasi
💻 **Coding**	Claude Code, Copilot	Implementasi
🧪 **Testing**	Playwright MCP	Pengujian otomatis
🤖 **Agents**	OpenClaw, Codex CLI	Task end-to-end via chat

✨ Hype vs Reality

> "AI will replace developers"

- ✅ AI **mempercepat** pekerjaan repetitif & eksplorasi solusi
- ❌ AI **belum bisa** menggantikan pemahaman arsitektur
- ❌ AI **sering salah** di edge case dan konteks bisnis
- ❌ AI **butuh arahan** jelas dari manusia yang paham konteks

⚠️ Jebakan Umum

- 🙈 **Copy-paste tanpa review** → bug tersembunyi
- 🍝 **Tanpa arsitektur** → sulit di-maintain
- 🔓 **Security blind spots** → vulnerability lolos
- 💣 **Over-confidence** → "AI yang bikin, pasti benar"

👥 Siapa yang Ada di Ruangan Ini?

Snapshot Peserta EDC Jogja

- 🏢 **Mayoritas sektor TIK** (25/49), **39/49 dari DIY**
- 👨‍💻 Terbesar: **Software & Web Developer** (14)
- 💼 Tapi juga ada **founder, business owner, IT manager, marketer, mahasiswa**
- 🌱 Usia dominan: **22-35 tahun**

👉 **Fokus: AI untuk kolaborasi lintas fungsi, bukan cuma ngoding cepat**

🗣️ Cerita Nyata: Yang Berhasil

- Migrasi Remix → React Router v7 — AI handle 80% boilerplate
- Refactor styling ke Tailwind — konsisten di 30+ komponen
- Personal AI army (Project Transformers) — 7 specialized agents, masing-masing punya domain sendiri, context tidak bocor antar agent
- Slide ini: draft pertama via Claude Web + Wheeljack (OpenClaw)

🗣️ Cerita Nyata: Yang Gagal

- AI generate test yang pass tapi **tidak test apa-apa** (assertion kosong)
- AI "fix" bug di satu tempat, tapi **break 3 tempat lain**
- AI bikin copy meyakinkan, tapi **tone dan audiens-nya meleset**
- Claire Vo (LaunchDarkly): OpenClaw **hapus family calendar** di percobaan pertama
- PR storm: arms race di kantor, **kualitas review turun** jika tidak disiplin

💡 Jadi Gimana?

- 🛡️ **Trust, but Verify** — selalu review
- 🧪 **Automate checks** — biarkan mesin bantu verifikasi
- 📋 **Mulai dari spec** — tujuan dulu, baru eksekusi
- 🧠 **Pahami output-nya** — jangan jadi passenger

2) Coverage & Control

📊 Coverage vs Control

	Coverage Rendah	Coverage Tinggi
Control Tinggi	🐢 Lambat tapi aman	🚀 Ideal
Control Rendah	💤 Tidak efisien	🔥 Chaos

👉 **Goal: Coverage Tinggi + Control Tinggi**

🚦 Level Kontrol Penggunaan AI

🟢 **Assistive** — AI menyarankan, manusia memutuskan
Autocomplete, grammar check, inline suggestions

🟡 **Semi-Autonomous** — AI kerjakan, manusia review
Draft copy, generate function, buat proposal

🔴 **Autonomous** — AI end-to-end, manusia verifikasi akhir
Prototype halaman, generate report

🧭 Framework: Kapan & Seberapa Besar

Situasi	Coverage	Control	Contoh
🎨 Prototyping	⬆️ Tinggi	⬇️ Rendah	Bolt.new, v0
📢 Marketing	⬆️ Tinggi	↔️ Sedang	Copy draft, content plan
💼 Business	↔️ Sedang	⬆️ Tinggi	Riset, ringkasan, proposal
🏗️ Production	↔️ Sedang	⬆️ Tinggi	Claude Code + Review
🔒 Security-critical	⬇️ Rendah	⬆️ Sangat Tinggi	Manual + AI assist

🔑 Kamu Tetap Pilot-nya

- 👨‍✈️ AI adalah **co-pilot**, bukan captain
- 🧠 Pahami keputusan pentingnya
- ✅ Yang bertanggung jawab tetap **kamu**
- ⚠️ Mid-career engineers paling terdampak — bukan junior (Simon Willison, Lenny's Podcast)

3) Prompting & Interaction Strategy

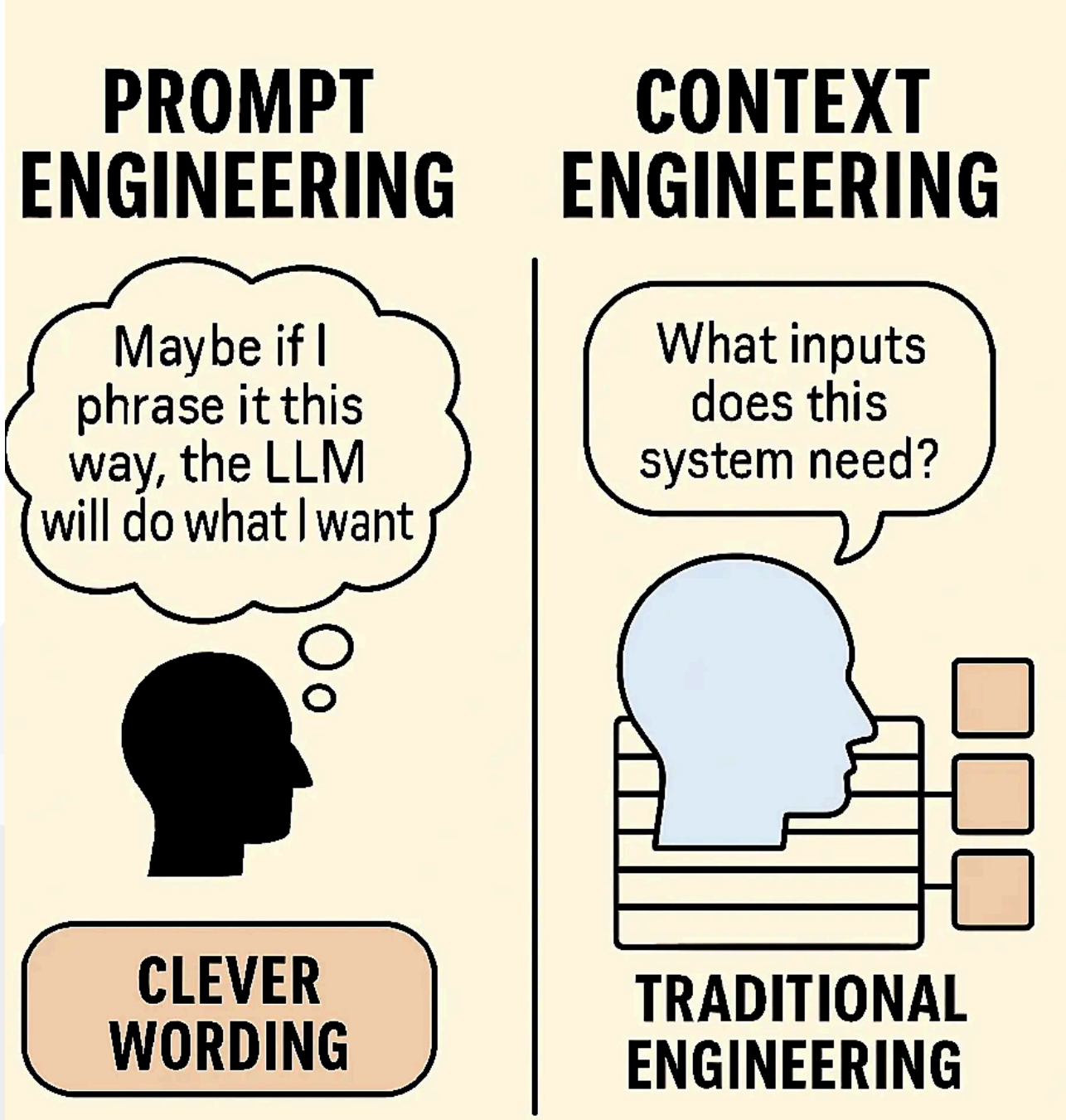
⌨️ Prompting 101

Prompt yang Baik = Hasil yang Baik

- 🎯 **Spesifik** — detail, bukan "buatkan sesuatu"
- 📋 **Kasih konteks** — file, constraint, target audiens
- 🖼️ **Kasih contoh** — tunjukkan format yang dimau
- 🔄 **Iterasi** — perbaiki berdasarkan hasil

🧠 Context Engineering > Prompt Engineering

Bukan hanya **cara bertanya**, tapi **informasi apa yang tersedia**



🧠 5 Jenis Context

- ⌨️ **User Input** — prompt kamu
- 📚 **Knowledge** — docs, riset, transcript
- 💬 **Conversation** — konteks sebelumnya
- 🛠️ **Tool Outputs** — hasil dari tools
- 📂 **Project Memory** — keputusan & artefak yang terus hidup

👉 **Semakin lengkap context, semakin baik output AI**

👥 Role-Based Interaction

Gunakan AI Sesuai Perannya

🏗️ **Architect** — "Desain arsitektur fitur X dengan constraint Y"

💻 **Implementer** — "Implementasikan function X sesuai spec ini"

🔍 **Reviewer** — "Review kode ini untuk security dan performance"

👥 Role-Based Interaction (lanjutan)

Bukan Hanya untuk Developer

📝 **Writing Partner** — "Draft landing page untuk UMKM teknologi di Jogja"

💼 **Business Analyst** — "Ringkas interview customer jadi 5 insight dan 3 aksi"

👉 **Sesuaikan peran AI dengan kebutuhan fungsi kamu**

📂 AI sebagai Second Brain

Insight dari Lenny's Newsletter

- 🧠 **Context is the new interface**
- 📚 Bukan cuma prompt — docs, riset, feedback, keputusan
- 🔍 Context tersimpan rapi → bukan cuma ingat lebih banyak, tapi **berpikir lebih jernih**
- 👥 Berlaku untuk semua fungsi, bukan cuma developer

🔄 Iterasi, Bukan Sekali Jadi

1. 1️⃣ **Prompt awal** → draft pertama
2. 2️⃣ **Review** → identifikasi kekurangan
3. 3️⃣ **Refine** → tambah constraint/context
4. 4️⃣ **Ulangi** sampai kualitas sesuai

💡 **Tips:** Gunakan `plan mode` untuk hemat token

⚡ Prototype dalam Menit, Bukan Minggu

Insight dari Lenny's Newsletter

- 🚀 Prototype interaktif bisa jadi dalam **hitungan menit**
- 💬 Uji ide **sebelum commit build penuh**
- 🎨 Untuk **semua fungsi** — samakan persepsi sebelum eksekusi
- ⚠️ Mulai kecil, iterasi bertahap

👉 **Prototype = alat belajar tercepat**

📜 CLAUDE.md = Autopilot Rules

Konsistensi Output Otomatis

```
# CLAUDE.md
## Git Conventions
- Do not include co-authorship in commit messages

## Architecture
- React Router v7 with file-based routing
- Tailwind CSS for styling
- TypeScript for type safety
```

👉 AI membaca ini dan mengikuti konvensi project

4) Structured AI Coding

🪜 Jangan Langsung Coding!

1. 📋 **Audience & goal** — untuk siapa, dan apa hasil yang dibutuhkan?
2. 🔄 **Flow** — bagaimana user atau tim akan memakainya?
3. 🏗️ **Approach** — bagaimana cara terbaik mengerjakannya?
4. 💻 **Implementation** — baru mulai eksekusi

🔄 Tanpa vs Dengan Struktur

❌ Tanpa Struktur

```
"Buatkan fitur authentication untuk app saya"
→ Random auth flow, missing edge cases
```

✅ Dengan Struktur

```
Requirement: Login via email/password + OAuth
Flow: Login → Validate → Session → Redirect
Constraints: Rate limiting, CSRF protection
→ AI delivers spec-compliant implementation
```

🧠 Prinsip Dasar

- ⚡ **Cepat tanpa struktur** = technical debt
- 🛡️ **Output tanpa review** = liability
- 📈 **Struktur + AI** = sustainable velocity

Mantra:

🎯 Define → 🧪 Test → 👀 Review

📝 Brief yang Terstruktur

Audience → Goal → Flow → Approach → Eksekusi

- 📝 **requirements** — kebutuhan yang bisa dicek
- 🎨 **design** — pendekatan dan alur kerja
- 📋 **tasks** — rencana yang trackable

👉 **AI bekerja lebih baik kalau brief-nya jelas**

5) Maintaining Craftsmanship

💎 Kualitas Tetap Nomor Satu

- 🔍 **Review setiap output** — jangan auto-accept
- 🧪 **Automate** — unit test, integration, E2E
- 📋 **Standar jelas** — checklist, style guide, acceptance criteria
- 👥 **Peer review** — mata manusia tetap penting

🔧 3 Pola Agentic Engineering

dari Simon Willison (Lenny's Podcast)

Pola	Cara Kerja	Contoh Nyata
🔴🟢 **Red/Green TDD**	Tulis test dulu, AI implementasi	Manusia malas TDD — AI tidak
📂 **Templates**	Pola di repo = panduan AI	Existing patterns, CLAUDE.md
📦 **Hoarding**	Kumpulkan contoh → AI kombinasi untuk kasus baru	Makin banyak pola, makin kreatif AI

🔍 AI untuk Review

- 👃 **Code smell** — duplicated logic, god functions
- ⚠️ **Anti-pattern** — circular deps, tight coupling
- 🐌 **Inefficiency** — N+1 queries, unnecessary re-renders
- 🔒 **Security** — injection, XSS, exposed secrets

🔄 Refactoring dengan AI

- 🎯 **Tentukan target** — pattern baru, bukan "perbaiki semua"
- 🧩 **Batch by scope** — satu modul / satu pattern per iterasi
- 🧪 **Test dulu, refactor kemudian** — pastikan ada safety net
- ✅ **Contoh nyata:** Refactor styling ke Tailwind — 30+ komponen, konsisten

🔒 Security: Manusia = Final Gatekeeper

- 🛡️ Validasi **vulnerability** — injection, insecure API, deps
- ⛔ Jangan blindly trust AI untuk security
- ⚠️ **Lethal trifecta** — private data + untrusted content + external comms
- 🤖 AI boleh **scan**, tapi manusia yang **approve**

⚙️ System Thinking

- 🧩 **Konsistensi** — satu pattern, seluruh produk
- ⛔ Hindari **over-fragmentation**
- 🔗 **Pahami impact** setiap perubahan
- 📚 **Docs** — AI bantu generate, manusia review

✅ Do's

- 🔍 **Review** setiap output
- 📋 **Konteks lengkap** — CLAUDE.md, spec, constraint
- 🧪 **Test + evals** otomatis
- 🔄 **Iterasi** bertahap
- 🧠 **Pahami** apa yang di-generate
- 📜 **Rules file** untuk konsistensi

❌ Don'ts

- 🙈 **Copy-paste tanpa baca**
- ❓ **Prompt ambiguous** — garbage in, garbage out
- 👀 **Vibe check doang** — bukan evaluasi
- 💻 **"Works on my machine"** — test di CI
- 🔮 **Expect sempurna sekali jadi**
- 😴 **Jadi passenger** di output sendiri
- 🚀 **Mulai tanpa context**

👀 Demo Results

🏆 Waktunya Cek Hasil!

Yang kita minta tadi:

> Filter by category di halaman `/talks` — Conferences, Podcasts, Meetups

👉 Buka terminal → cek status Claude Code
👉 Review PR yang masuk di GitHub
👉 Merge PR → baru zainf.dev/talks terupdate

💡 Key Takeaways

1. 🤖 **Agentic Engineering** — bukan vibe coding, tapi specify → verify → iterate
2. 📊 **Coverage & control** harus seimbang
3. 🧠 **Context > prompt** — termasuk audiens dan tujuan
4. 📋 **Struktur dulu** — audience → goal → flow → approach
5. 💎 **Kualitas tetap nomor satu** — review, test, verify
6. ⚙️ **System thinking** — konsistensi di seluruh produk

🙏 Terima Kasih!

🖥️ https://zainf.dev/agentic-engineering

🔗 https://github.com/zainfathoni/zainf

Ada pertanyaan? ✋

Referensi

- Context Engineering — Addy Osmani
- How to Make Your Tech Stack AI-Friendly — Refactoring · Kiro Specs — AWS
- Lenny's Podcast — How OpenClaw Changed My Life (Claire Vo) · An AI State of the Union (Simon Willison)
- Lenny's Newsletter — *PM second brain with ChatGPT · AI prototyping for PMs*
- What is Agentic Engineering? — Simon Willison
- Arms Race di Kantor · Building My Personal AI Army — Zain Fathoni
- OpenClaw — Agentic engineering platform